

Rockchip RK3506 Linux SDK Quick Start Guide

ID: RK-JC-YF-A27

Release Version: V1.1.0

Release Date: 2024-11-28

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2024 Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the basic usage of the RK3506 Linux SDK, aiming to help developers quickly understand and use the RK3506 SDK development package.

Target Audience

This document (this guide) is mainly applicable to the following engineers:

Technical Support Engineers

Software Development Engineers

System Support Status

SoC	Uboot Version	Kernel Version	Buildroot Version
RK3506	2017.9	6.1	2024.02

Revision Record

Date	Version	Author	Modification Description
2024-07-20	V0.0.1	Zack Huang	Initial version
2024-08-20	V0.1.0	Zack Huang	Beta version
2024-09-20	V1.0.0	Zack Huang	Release version
2024-11-28	V1.1.0	Zack Huang	Added the description of memory configuration

Table of Contents

Rockchip RK3506 Linux SDK Quick Start Guide

1. Precompiled SDK Images
2. Development Environment Setup
 - 2.1 Preparing the Development Environment
 - 2.2 Installing Libraries and Toolkits
 - 2.2.1 Checking and Upgrading the Host's Python Version
 - 2.2.2 Checking and Upgrading the `make` Version of the Host
 - 2.2.3 Checking and Upgrading the LZ4 Version of the Host
3. Docker Environment Setup
4. Software Development Guide
 - 4.1 Development Guide
 - 4.2 SoC Document
 - 4.3 Buildroot Development Guide
 - 4.4 Software Update Log
5. Hardware Development Guide
6. SDK Project Directory Overview
7. Introduction to Cross-Compilation Toolchain for SDK
 - 7.1 U-Boot and Kernel Compilation Toolchain
 - 7.2 Buildroot Toolchain
 - 7.2.1 Configuring the Compilation Environment
 - 7.2.2 Packaging Toolchain
8. SDK Compilation Instructions
 - 8.1 Checking SDK Compilation Commands
 - 8.2 SDK Board-Level Configuration
 - 8.3 Customize Configuration of the SDK
 - 8.4 Fully Automatic Compilation
 - 8.5 Module Compilation
 - 8.5.1 U-Boot Compilation
 - 8.5.2 Kernel Compilation
 - 8.5.3 Recovery Compilation
 - 8.5.4 Buildroot Compilation
 - 8.5.4.1 Buildroot Module Compilation
 - 8.5.5 Yocto Compilation
 - 8.6 Firmware Packaging
9. Flashing Instructions
 - 9.1 Windows Flashing Instructions
 - 9.2 Linux Flashing Instructions
 - 9.3 System Partition Description
10. Product module development instructions
 - 10.1 Introduction to audio noise reduction algorithm debugging
 - 10.1.1 Clarify the type and specification indicators of the current product
 - 10.1.2 Introduction to audio algorithm test related knowledge points
 - 10.1.3 Audio algorithm test scenario
 - 10.1.4 Audio channel test
 - 10.1.4.1 Audio playback maximum undistorted signal test
 - 10.1.4.2 Audio sweep recording and playback test
 - 10.1.5 Audio algorithm debugging
 - 10.2 Firmware size
 - 10.3 OTG-ID and OTG-VBUS-DET configuration Description
 - 10.3.1 The OTG ID configure
 - 10.4 Video software decoding
 - 10.5 MCU
 - 10.6 Flexbus(DVP, High Speed IO)

- 10.7 The RS485 hardware scheme automatically switches
- 10.8 Real-time optimization
- 10.9 EtherCat
- 10.10 CAN
- 10.11 DSMC
- 10.12 LVGL Demo
 - 10.12.1 Enable Functions of RK_DEMO
- 10.13 Display
- 10.14 Read-write file system
- 10.15 How to use eMMC Storage Media

1. Precompiled SDK Images

Developers can bypass the process of compiling the entire operating system from source code by using the precompiled RK3506 Linux SDK image, which allows for a quick start in development and related assessments and comparisons. This can reduce the waste of development time and cost due to compilation issues.

The firmware can be downloaded from the public address [Click here for SDK firmware download](#).

Firmware path: **Universal Linux SDK Firmware -> Linux 6.1 -> RK3506**

For those who need to modify the SDK code or require a quick start, please refer to the following sections.

2. Development Environment Setup

2.1 Preparing the Development Environment

We recommend using a system with Ubuntu 22.04 or a higher version for compilation. Other Linux versions may require corresponding adjustments to the software packages. In addition to system requirements, there are other hardware and software requirements.

Hardware Requirements: 64-bit system with hard disk space greater than 40GB. If you are performing multiple builds, you will need more hard disk space.

Software Requirements: System with Ubuntu 22.04 or a higher version.

2.2 Installing Libraries and Toolkits

When developing for devices using the command line, you can install the necessary libraries and tools required for compiling the SDK by following these steps.

Use the following `apt-get` commands to install the libraries and tools needed for subsequent operations:

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

Note:

The installation command is suitable for Ubuntu 22.04. For other versions, please use the corresponding installation command based on the package name. If you encounter errors during compilation, you can install the corresponding software packages based on the error messages. Specifically:

- Python requires the installation of version 3.6 or higher, with version 3.6 used as an example here.
- Make requires the installation of version 4.0 or higher, with version 4.2 used as an example here.

- LZ4 requires the installation of version 1.7.3 or higher.
- Compiling Yocto requires a VPN network.

2.2.1 Checking and Upgrading the Host's Python Version

The method to check and upgrade the host's Python version is as follows:

- Check the host's Python version

```
$ python3 --version
Python 3.10.6
```

If the requirement of Python version ≥ 3.6 is not met, you can upgrade by the following method:

- Upgrade to the new version of `Python 3.6.15`

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 Checking and Upgrading the `make` Version of the Host

The method to check and upgrade the `make` version of the host is as follows:

- Checking the `make` version of the host

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- Upgrading to the newer version of `make 4.2`

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 Checking and Upgrading the LZ4 Version of the Host

The method to check and upgrade the LZ4 version of the host is as follows:

- Check the LZ4 version of the host

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- Upgrade to the newer version of LZ4

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

3. Docker Environment Setup

To assist developers in quickly completing the complex preparation of the development environment mentioned above, we also provide a cross-compiler Docker image for clients to rapidly verify, thereby reducing the time required to build the compilation environment.

Before using the Docker environment, you can refer to the following document for operations:

<SDK>/docs/en/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_EN.pdf.

The verified systems are as follows:

Distribution Version	Docker Version	Image Loading	Firmware Compilation
Ubuntu 22.04	24.0.5	pass	pass
Ubuntu 21.10	20.10.12	pass	pass
Ubuntu 21.04	20.10.7	pass	pass
Ubuntu 18.04	20.10.7	pass	pass
Fedora 35	20.10.12	pass	NR (not run)

The Docker image can be obtained from the URL [Docker Image](#).

4. Software Development Guide

4.1 Development Guide

To assist development engineers in quickly becoming proficient with SDK development and debugging, the "Rockchip_Developer_Guide_Linux_Software_EN.pdf" is released along with the SDK. It can be obtained in the `docs/en/RK3506` directory and will be continuously improved and updated.

4.2 SoC Document

To assist development engineers in quickly getting started with the development and debugging of RK3506, the "Rockchip RK3506 Datasheet V0.1-20240516.pdf" is released with the SDK. It can be obtained in the `docs/en/RK3506/Datasheet` directory and will be continuously improved and updated.

4.3 Buildroot Development Guide

To assist development engineers in quickly getting started with the development and debugging of the Buildroot system, the development guide "Rockchip_Developer_Guide_Buildroot_EN.pdf" is released along with the SDK. It can be obtained in the `docs/en/Linux/System` directory and will be continuously improved and updated.

4.4 Software Update Log

- Software release version upgrades can be viewed through the project XML. The detailed method is as follows:

```
.repo/manifests$ realpath rk3506_linux6.1_release.xml
# For example: The printed version number is v0.0.1, and the update time is
20240720
# <SDK>/repo/manifests/release/rk3506_linux_alpha_v0.0.1_20240720.xml
```

- The updated content of the software release version can be viewed through the project text, please refer to the project directory:

```
<SDK>/repo/manifests/RK3506_Linux6.1_SDK_Note.md
or
<SDK>/docs/cn/RK3506/RK3506_Linux6.1_SDK_Note.md
```

- On the board, the version information can be checked in the following way:

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.1-stan-rkr2
ID=buildroot
VERSION_ID=2024.02
PRETTY_NAME="Buildroot 2024.02"
OS="buildroot"
RK_BUILD_INFO="xxx Thu Apr 18 17:59:46 CST 2024 - rockchip_rk3506"
```


5. Hardware Development Guide

Hardware development document can refer to the user guide in the project directory:

RK3506 Hardware Design Guide:

```
<SDK>/docs/cn/RK3506/Hardware/
```

6. SDK Project Directory Overview

The SDK project directory includes directories such as buildroot, app, kernel, u-boot, device, docs, external, prebuilts, etc. It uses a manifest to manage the repository and the repo tool to manage each directory or its corresponding git projects.

- app: Contains upper-layer application APPs, mainly some application demos.
- buildroot: The root file system developed based on Buildroot (2024).
- device/rockchip: Contains SoC-level board configurations as well as scripts and files for SDK compilation and firmware packaging.
- docs: Contains general development guidance documents, SoC platform-related documents, Linux system development-related documents, and other reference documents.
- external: Contains third-party related repositories, including display, audio and video, camera, network, security, etc.
- kernel: Contains the code for Kernel development.
- output: Contains information about each firmware generation, compilation information, XML, host environment, etc.
- prebuilts: Contains cross-compilation toolchains.
- rkbin: Contains Rockchip-related binaries and tools.
- rockdev: Contains compiled output firmware, actually, it is soft link to `output/firmware`.
- tools: Contains commonly used tools for Linux and Windows operating systems.
- u-boot: Contains U-Boot code developed based on the v2017.09 version.
- rtos: Contains rtos related source code.
- yocto: Contains rtos related source code and script.

7. Introduction to Cross-Compilation Toolchain for SDK

Considering that the Rockchip Linux SDK is currently compiled only in the Linux PC environment, we have also provided the cross-compilation toolchain for Linux only. The prebuilt toolchain in the prebuilt directory is for use with U-Boot and Kernel. Rootfs requires the corresponding toolchain, or a third-party toolchain for compilation.

7.1 U-Boot and Kernel Compilation Toolchain

The SDK prebuilts directory contains cross-compilation tools currently used for U-Boot and Kernel compilation, as follows:

Directory	Description
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf	gcc arm 10.3.1 32-bit toolchain

7.2 Buildroot Toolchain

7.2.1 Configuring the Compilation Environment

To compile individual modules or third-party applications, a cross-compilation environment must be set up. For instance, for the RK3506, the cross-compilation tools are located in the `buildroot/output/rockchip_rk3506/host/usr` directory. It is necessary to set the `bin/` directory of the tools and the `arm-buildroot-linux-gnueabihf/bin/` directory as environment variables. Execute the script for automatic configuration of the environment variables in the top-level directory:

```
source buildroot/envsetup.sh rockchip_rk3506
```

Enter the command to check:

```
cd buildroot/output/rockchip_rk3506/host/usr/bin
./arm-linux-gcc --version
```

At this point, the following information will be printed:

```
arm-linux-gcc.br_real (Buildroot linux-6.1-stan-rkr3-40-gcec74d50ae-dirty) 12.3.0
```

7.2.2 Packaging Toolchain

Buildroot supports the packaging of the built-in toolchain into a compressed archive for standalone compilation by third-party applications. For detailed information on how to package the toolchain, please refer to the Buildroot official documentation:

```
buildroot/docs/manual/using-buildroot-toolchain.txt
```

In the SDK, you can directly run the following command to generate the toolchain package:

```
./build.sh bmake:sdk
```

The generated toolchain package is located in the `buildroot/output/*/images/` directory, named `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`, for users who have the need. After extraction, the path to `gcc` will be:

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

8. SDK Compilation Instructions

The SDK can be configured and compiled for specific functionalities using `make` or `./build.sh` with target parameters. For detailed instructions, please refer to the compilation guide in `device/Rockchip/common/README.md`.

To ensure a smooth update process for the SDK each time, it is recommended to clean the previous build artifacts before updating, in order to avoid potential compatibility issues or compilation errors that may arise from using old compilation artifacts with a new version of the SDK. To clean these artifacts, simply run the command `./build.sh cleanall`.

One-Click Compilation

SoC	Type	Reference Model	One-Click Compilation
RK3506	Development Board	RK3506G EVB1	<code>./build.sh lunch:rockchip_rk3506_g_evb1_defconfig && ./build.sh</code>
RK3506	Development Board	RK3506B EVB1	<code>./build.sh lunch:rockchip_rk3506_b_evb1_defconfig && ./build.sh</code>
RK3506	Demo Board	RK3506G DEMO	<code>./build.sh lunch:rockchip_rk3506_g_demo_defconfig && ./build.sh</code>

Note:

- Rootfs Compilation:**
The default is to build Buildroot system. If other systems are required, set the corresponding environment variables.
- Kernel and U-Boot Compilation:** It is necessary to specify the toolchain.
- For example, the SDK's built-in 32-bit toolchain:
`export CROSS_COMPILE=./prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-`

8.1 Checking SDK Compilation Commands

`make help`, for example:

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconfig           - Same as oldconfig, but quietly, additionally update
deps
olddefconfig        - Same as synconfig but sets new symbols to their
default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

The actual execution of make is `./build.sh`

You can also compile related features by running `./build.sh <target>`, and you can view the specific compilation commands through `./build.sh help`.

```
./build.sh -h

##### Rockchip Linux SDK #####

Manifest: rk3506_linux6.1_beta_20240820_v0.1.0.xml

Log colors: message notice warning error fatal1

sage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]      choose chip
defconfig[:<config>]         choose defconfig
*_defconfig                  switch to specified defconfig
    available defconfigs:
    rockchip_defconfig
olddefconfig                 resolve any unresolved symbols in .config
savedefconfig               save current config to defconfig
menuconfig                  interactive curses-based configurator
config                      modify SDK defconfig
print-parts                 print partitions
list-parts                  alias of print-parts
mod-parts                   interactive partition table modify
edit-parts                  edit raw partitions
new-parts:<offset>:<name>:<size>... re-create partitions
insert-part:<idx>:<name>[:<size>] insert partition
del-part:(<idx>|<name>)       delete partition
move-part:(<idx>|<name>):<idx> move partition
rename-part:(<idx>|<name>):<name> rename partition
resize-part:(<idx>|<name>):<size> resize partition
misc                        pack misc image
kernel-6.1[:dry-run]        build kernel 6.1
kernel[:dry-run]            build kernel
recovery-kernel[:dry-run]   build kernel for recovery
modules[:dry-run]           build kernel modules
linux-headers[:dry-run]     build linux-headers
```

kernel-config[:dry-run]	modify kernel defconfig
kconfig[:dry-run]	alias of kernel-config
kernel-make[:<arg1>:<arg2>]	run kernel make
kmake[:<arg1>:<arg2>]	alias of kernel-make
wifibt[:<dst dir>[:<chip>]]	build Wifi/BT
amp	build and pack amp system
buildroot-config[:<config>]	modify buildroot defconfig
bconfig[:<config>]	alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]	run buildroot make
bmake[:<arg1>:<arg2>]	alias of buildroot-make
rootfs[:<rootfs type>]	build default rootfs
buildroot	build buildroot rootfs
yocto	build yocto rootfs
debian	build debian rootfs
recovery	build recovery
security-createkeys	create keys for security
security-misc	build misc with system encryption key
security-ramboot	build security ramboot
security-system	build security system
loader[:dry-run]	build loader (u-boot)
uboot[:dry-run]	build u-boot
u-boot[:dry-run]	alias of uboot
uefi[:dry-run]	build uefi
extra-parts	pack extra partition images
firmware	pack and check firmwares
edit-package-file	edit package-file
edit-ota-package-file	edit package-file for OTA
updateimg	build update image
ota-updateimg	build update image for OTA
all	build images
release	release images and build info
all-release	build and release images
shell	setup a shell for developing
cleanall	cleanup
clean[:module[:module]]...	cleanup modules
available modules:	
all	
amp	
config	
extra-parts	
firmware	
kernel	
loader	
misc	
recovery	
rootfs	
security	
updateimg	
post-rootfs <rootfs dir>	trigger post-rootfs hook scripts
help	usage

Default option is **'all'**.

8.2 SDK Board-Level Configuration

Navigate to the project directory `<SDK>/device/Rockchip/rk3506`:

Board-Level Configuration	Description
rockchip_defconfig	Default configuration, which will be soft linked to the RK3506G EVB1 development board configuration
rockchip_rk3506_g_evb1_defconfig	This configuration applies to the RK3506G EVB1 AIOT board that uses the linux solution
rockchip_rk3506_b_evb1_defconfig	This configuration applies to the RK3506B EVB1 AIOT board that uses the linux solution
rockchip_rk3506_g_demo_defconfig	This configuration applies to the RK3506G DEMO AIOT board that uses the linux solution
rockchip_rk3506_g_evb1_amp_defconfig	This configuration applies to the RK3506G EVB1 AIOT board that uses the amp solution CPU0~CPU1: Linux CPU2: RT_Thread
rockchip_rk3506_g_evb1_smp_defconfig	This configuration applies to the RK3506G EVB1 AIOT board that uses the smp solution CPU0~CPU2: RT_Thread

Note: Please refer to the documentation for using smp and amp solution:

`SDK\docs\en\Common\AMP\Rockchip_Developer_Guide_AMP_EN.pdf`

`make lunch` or `./build.sh lunch` can be used to configure:

```
$. /build.sh lunch

You're building on Linux
Lunch menu...pick a combo:

1. rockchip_rk3506_b_evb1_defconfig
2. rockchip_rk3506_g_demo_defconfig
3. rockchip_rk3506_g_evb1_defconfig
...
Which would you like? [1]:
```

Most customers use EVB1 boards, which are suitable for general application scenarios.

Other feature configurations can be set through `make menuconfig` to configure related properties.

8.3 Customize Configuration of the SDK

The SDK can be configured through `make menuconfig`, and the main configurable components are as follows:

```
(rockchip_rk3506_g_evbl_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto) --->
[*] Loader (U-Boot) --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image) --->
    Boot (Android-style boot image) --->
[*] Recovery (based on Buildroot) --->
    *** Security feature depends on buildroot rootfs ***
    Extra partitions (oem, userdata, etc.) --->
    Firmware (partition table, misc image, etc.) --->
[*] Update (Rockchip update image) --->
    Others configurations --->
```

- Rootfs: The Rootfs here represents the "root file system," where you can choose different root file system configurations like Buildroot.
- Loader (U-Boot): This is the configuration for the bootloader, typically U-Boot, which is used to initialize the hardware and load the main operating system.
- AMP: A multi-core heterogeneous boot solution suitable for application scenarios requiring real-time performance.
- Kernel: Configure the kernel options to customize the Linux kernel according to your hardware and application needs.
- Boot: configure the support format for the Boot partition.
- Recovery (based on Buildroot): This is the configuration for the recovery environment based on Buildroot, used for system recovery and upgrades.
- PCBA test (based on Buildroot): This is the configuration for a PCBA test environment based on Buildroot.
- Security: Security features are enabled, including Secureboot methods, Optee storage methods, and burn-in keys.
- Extra partitions: Used to configure additional partitions.
- Firmware: Configure firmware-related options.
- Update (Rockchip update image): Used to configure options for the Rockchip complete firmware.
- Others configurations: Other additional configuration options.

The `make menuconfig` configuration interface provides a text-based user interface for selecting and configuring various options.

After the configuration is complete, use the `make savedefconfig` command to save these configurations, so that the customized compilation will proceed according to these settings.

Through the above config, you can choose different Rootfs/Loader/Kernel configurations for various customized compilations, allowing you to flexibly select and configure system components to meet specific requirements.

8.4 Fully Automatic Compilation

To ensure that each update of the Software Development Kit (SDK) proceeds smoothly, it is recommended to clean up the previous compilation artifacts before updating. This step can prevent potential compatibility issues or compilation errors, as old compilation artifacts may not be suitable for the new version of the SDK. To clean these artifacts, you can simply run the command `./build.sh cleanall`.

Navigate to the project's root directory and execute the following command to automatically complete all compilations:

```
./build.sh all # Compiles only module code (u-Boot, kernel, Rootfs, Recovery)
              # Further execute `./build.sh ./mkfirmware.sh` for firmware
packaging

./build.sh    # Compiles module code (u-Boot, kernel, Rootfs, Recovery)
              # Packages into a complete update.img upgrade package
              # All compilation information is copied and generated in the out
directory
```

The default is Buildroot, but you can specify a different rootfs by setting the environment variable

```
RK_ROOTFS_SYSTEM.
```

8.5 Module Compilation

8.5.1 U-Boot Compilation

```
./build.sh uboot
```

For the RK3506 platform, the SDK provides relevant tailoring and customization configurations for uboot, as follows:

config	Instructions
rk3506_defconfig	RK3506 basic configuration
rk3506_tb.config	RK3506 fast boot sub-configuration
rk-amp.config	RK3506 amp sub-configuration

Usage: basic configuration + sub-configuration

Take the RK3506G EVB1 development board as an example: To use the quick start solution, you need to edit the following files: SDK/device/rockchip/rk3506/rockchip_rk3506_g_evb1_defconfig:

```
RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
```



```
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y
```

If you don't use quick scheme, configuration file SDK/device/rockchip rk3506 / rockchip_rk3506_g_evb1_defconfig is as follows:

```
RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y
```

Major difference:

```
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb" #This is configured for using quick-start
uboot fragments
RK_UBOOT_CFG_FRAGMENTS="rk3506"    #This is the normal uboot configuration
```

8.5.2 Kernel Compilation

- Method One

```
./build.sh kernel
```

- Method Two

```
cd kernel
export CROSS_COMPILE=../prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-
arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-
make ARCH=arm rk3506_defconfig;
make ARCH=arm rk3506g-evb1-v10.img -j24
```

Note: `rk3506g-evb1-v10` can be replaced with the specific hardware DTS

For RK3506 platform, the SDK provides relevant tailoring and customization configurations for kernel, as follows:

config	Instructions
rk3506_defconfig	RK3506 basic configuration
rk3506-display.config	RK3506 display sub-config
rk3506-wifibt.config	RK3506wifi bt sub-config
rk3506_usbhost.config	RK3506 usb host sub-config
rk3506-usb-otg.config	RK3506 usb otg sub-config
rk3506-usb-peripheral.config	RK3506 usb peripheral sub-config
rk3506-ethercat.config	RK3506 usb ethercat sub-config

Usage: basic configuration + sub-configuration

Take the RK3506G EVB1 development board as an example: For example, you need to edit the following files: SDK/device/rockchip/rk3506/rockchip_rk3506_g_evb1_defconfig

```

RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y

```

8.5.3 Recovery Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of Recovery.

```
<SDK># ./build.sh recovery
```

After compilation, the recovery.img is generated in the Buildroot directory

```
output/rockchip_rk3506_recovery/images
```

Note: The recovery.img includes the kernel, so every time the Kernel is modified, Recovery needs to be repackaged and generated. The method to repack Recovery is as follows:

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
<SDK># make recovery-reconfigure
<SDK># cd -
<SDK># ./build.sh recovery
```

Note: Recovery is a non-essential feature, and some board-level configurations may not set it up.

8.5.4 Buildroot Compilation

For RK3506 platform, buildroot has relevant tailoring and customization configuration in SDK, as follows:

config	Instructions
rockchip_rk3506_defconfig	RK3506 Default configuration
rockchip_rk3506_tiny_defconfig	The RK3506 removed the configuration of the display part for some products without a display screen

Method of use: the device/rockchip/chip/rk3506 rockchip_rk3506_XXXX_defconfig modify:

```
diff --git a/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
b/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
index bc06b077..887c9e24 100644
--- a/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
+++ b/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
@@ -1,5 +1,6 @@
RK_ROOTFS_UBI=y
RK_WIFIBT_CHIP="AP6256"
+RK_BUILDROOT_CFG="rk3506_tiny"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
```

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh rootfs
```

After the compilation, different formats of the image are generated in the `output/rockchip_rk3506/images` directory under the Buildroot directory, with the default format being rootfs.ubi.

For more details, please refer to the Buildroot development documentation:

```
<SDK>/docs/en/Linux/System/Rockchip_Developer_Guide_Buildroot_EN.pdf
```

8.5.4.1 Buildroot Module Compilation

Set up configurations for different chips and target features via `source buildroot/envsetup.sh`

```
$ source buildroot/envsetup.sh
Top of tree: rk3506

You're building on Linux
Lunch menu...pick a combo:

49. rockchip_rk3506
50. rockchip_rk3506_recovery
...

Which would you like? [1]:
```

The default selection is `rockchip_rk3506`. Then enter the RK3506 Buildroot directory to start compiling the relevant modules.

The `rockchip_rk3506_recovery` is used for compiling the Recovery module.

For example, to compile the rockchip-test module, the common compilation commands are as follows:

Navigate to the buildroot directory

```
<SDK># cd buildroot
```

- Delete and recompile rockchip-test

```
buildroot# make rockchip-test-recreate
```

- Rebuild rockchip-test

```
buildroot# make rockchip-test-rebuild
```

- Delete rockchip-test

```
buildroot# make rockchip-test-dirclean
or
buildroot# rm -rf output/rockchip_rk3506/build/rockchip-test-master/
```

8.5.5 Yocto Compilation

Navigate to the root directory of the project and execute the following command to automatically complete the compilation and packaging of the Rootfs:

```
./build.sh yocto
```

After compilation, the rootfs.img is generated in the yocto directory under build/lastest.

The default username for login is root. For more information on Yocto, please refer to the [Rockchip Wiki](#).

FAQ:

- If you encounter the following issue during the compilation above:

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).  
Python can't change the filesystem locale after loading so we need a UTF-8  
when Python starts or things won't work.
```

Solution:

```
locale-gen en_US.UTF-8  
export LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8
```

Or refer to [setup-locale-python3](#)

- If you encounter git permission issues that cause compilation errors.

Due to the addition of the CVE-2022-39253 security detection patch in the newer version of git, if an older version of Yocto is required, poky should include the following to fix it:

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba  
Author: Robert Yang <liezhi.yang@windriver.com>  
Date:   Fri Mar 24 00:09:02 2023 -0700  
  
    bitbake: fetch/git: Fix local clone url to make it work with repo  
  
    The "git clone /path/to/git/objects_symlink" couldn't work after the  
    following  
    change:  
  
    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

Alternatively, you can also revert the git version on the PC to V2.38 or an earlier version, such as the following:

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev  
  
git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0  
cd git  
make git -j8  
make install  
sudo install -m 0755 git /usr/bin/git
```

8.6 Firmware Packaging

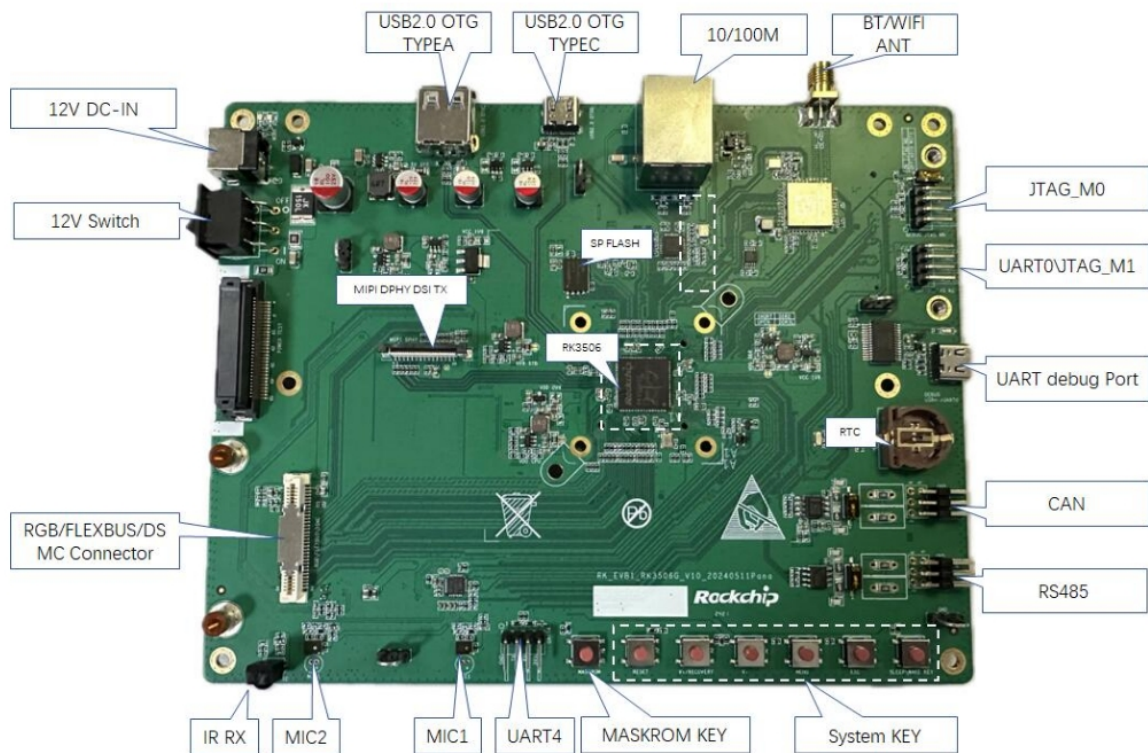
After compiling above components such as Kernel, U-Boot, Recovery, and Rootfs, navigate to the root directory of the project and execute the following command to automatically complete the packaging of all firmware into the `output/firmware` directory:

Firmware Generation:

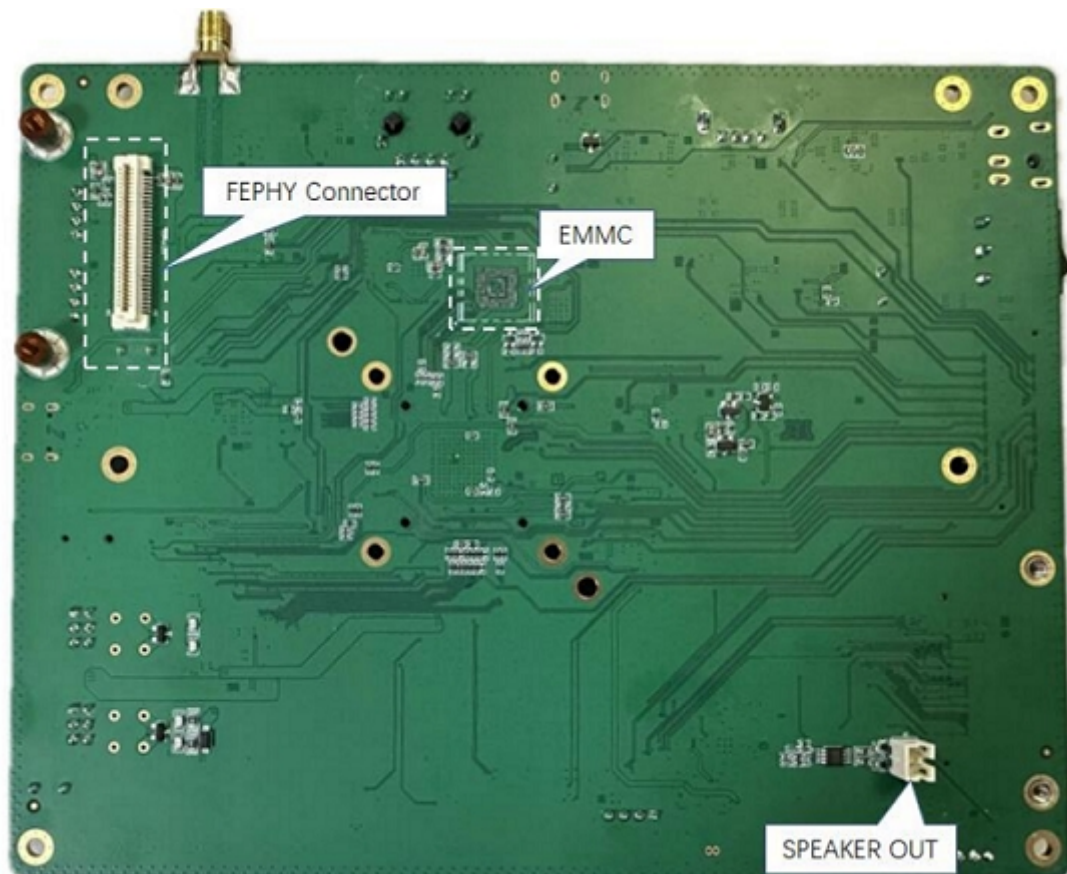
```
./build.sh firmware
```

9. Flashing Instructions

The interface layout of the TOP Surface of RK3506 EVB 1 development board is as follows:



The interface layout of the Bottom Surface of RK3506 EVB 1 development board is as follows:



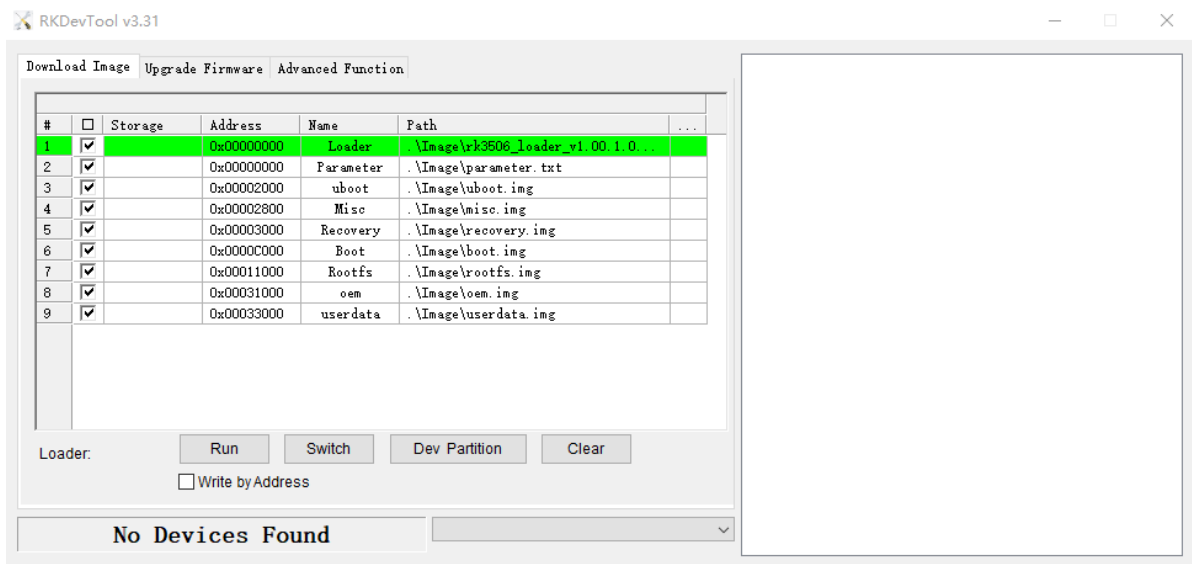
9.1 Windows Flashing Instructions

The SDK provides a Windows flashing tool (the tool version must be V3.28 or above), located in the root directory of the project:

```
tools/  
└─ windows/RKDevTool
```

As shown in the figure below, after compiling the corresponding firmware, the device needs to enter the MASKROM or BootROM flashing mode. Connect the USB download cable, hold the "MASKROM" button and press the reset button "RST" without releasing, then release to enter the MASKROM mode. Load the corresponding path of the compiled firmware and click "Execute" to flash. Alternatively, hold the "recovery" button without releasing and press the reset button "RST", then release to enter the loader mode for flashing. Below are the partition offsets for MASKROM mode and the flashing files.

(Note: The Windows PC needs to run the tool with administrator privileges to execute)



Note: Before flashing, the latest USB driver must be installed. For more details on the driver, see:

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

9.2 Linux Flashing Instructions

The flashing tool for Linux is located in the `tools/linux` directory (the `Linux_Upgrade_Tool` version must be V2.36 or above). Please ensure that your board is connected to MASKROM/loader rockusb. For example, if the firmware compiled is in the `rockdev` directory, the upgrade commands are as follows:


```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

Or upgrade the complete firmware after packaging:

```
sudo ./upgrade_tool uf rockdev/update.img
```

Or upgrade in the root directory when the device is running in MASKROM state:

```
./rkflash.sh
```

9.3 System Partition Description

Default Partition Description (Below is the RK3506 EVB Partition Reference)

- uboot Partition: For the uboot.img compiled from uboot.
- misc Partition: For misc.img, used by recovery.
- boot Partition: For boot.img compiled from the kernel.
- recovery Partition: For recovery.img compiled by recovery.
- backup Partition: Reserved, not in use at the moment.
- rootfs Partition: For rootfs.img compiled by buildroot
- oem Partition: For manufacturers to use, storing manufacturer's apps or data. Mounted at the /oem directory.
- userdata Partition: For apps to temporarily generate files or for users, mounted under the /userdata directory.

10. Product module development instructions

10.1 Introduction to audio noise reduction algorithm debugging

When faced with the need for audio parameter adjustment, most developers are accustomed to blindly adjusting the audio algorithm interface and parameters, while ignoring whether the front-end input signal is correct and undistorted. In fact, algorithm debugging only plays an auxiliary role at the end of the entire audio effect path. What really needs to be paid attention to is often **the acoustic structure of the algorithm front end, as well as hardware (components) indicators and other links**. Only when the signal obtained by the algorithm front end is not distorted, the algorithm can truly play its role effectively.

Therefore, the current product type and specification indicators can be clarified before parameter adjustment to achieve twice the result with half the effort.

10.1.1 Clarify the type and specification indicators of the current product

First, developers should understand the current product type and the demand for audio effects. For example: how many microphones and speakers are used for playback? Is the product type IPC, or conference pickup system, etc.? Is there a need for two-way calls? Usually IPC products do not require more than 2 microphones for pickup, while conference systems require more microphones. If it is a microphone array type in a multi-microphone scenario (such as a 4MIC topology, is it a ring microphone or a line microphone).

Secondly, determine the pickup distance range (such as 3 meters to 10 meters) and playback loudness required by the product design. The following information can be queried in the device manual:

- Is the microphone an analog microphone (mostly electret) or a digital microphone? Usually analog microphones have better effects on far-field pickup
- The SNR of the microphone (usually not less than 65dB), sensitivity, taking analog microphones as an example, for far-field pickup of more than 5 meters, it is not less than -32dBV
- The power and load of the speaker playback output, such as 2W@8ohm,
- Product structure design related picture information, such as MIC aperture, and the position, distance, distribution, etc. with SPK
- Use hardware or software to collect. Currently, software collection applications are mostly used

Next, you also need to determine the sampling rate of the recording and playback used by the product, For example, it is usually 16kHz or 8kHz. If you need to use AI noise reduction or crying detection, you need to use a sampling rate of 16kHz or above.

10.1.2 Introduction to audio algorithm test related knowledge points

Most users need to enable audio algorithms. The related modules mainly include ANR, which is used to eliminate environmental noise; and AEC (Acoustic Echo Cancellation) is enabled in call scenarios to eliminate the echo from the remote user during intercom calls. The following block diagram describes the data flow and processing effect of the classic echo cancellation algorithm when it is working:

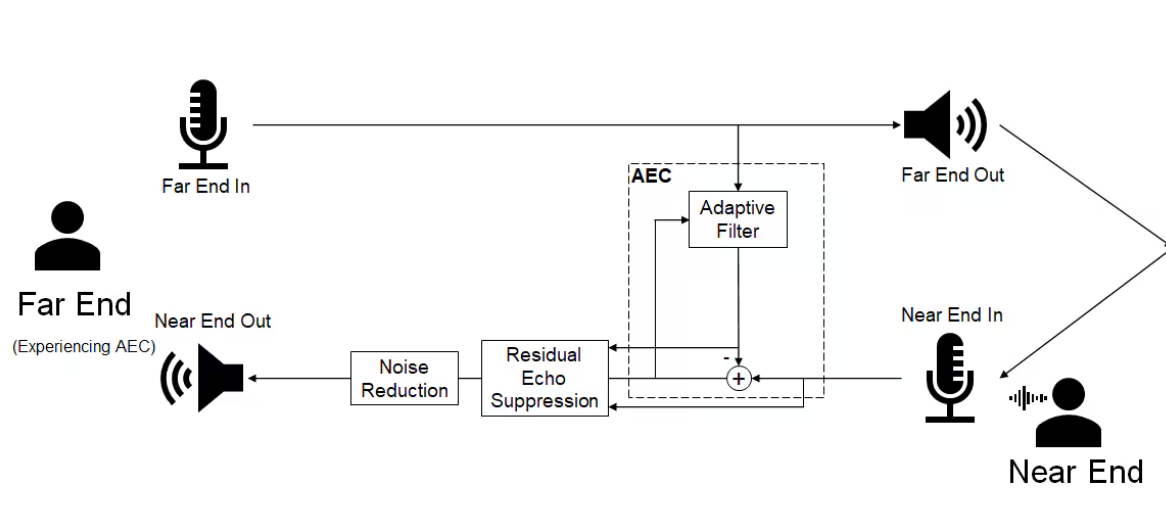


Figure Echo Cancellation Workflow

There are several terms that need to be clarified and unified:

- Near End: From the perspective of the machine under test, for example, the speaker speaks to the MIC of the machine, which we call "near-end input human voice". Then, this speaking voice is transmitted to the remote device through the network and played.

- Far End: refers to the remote device that communicates with the machine under test, such as the tester speaks to the MIC of the remote device, which is transmitted to the machine under test through the network and played. At the same time, this part of the data is also stored in another recording channel of the machine under test through the way of recollection (whether software recollection or hardware recollection) as a reference signal for echo cancellation.

10.1.3 Audio algorithm test scenario

Users usually encounter the following usage scenarios when talking on the intercom:

- **Near end does not speak, far end speaks**

In this case, the system is in the **far end single talk** state, and the far end sound is played through the near end speaker and then collected back by the microphone (microphone input includes echo and noise). This is usually the first test and concern of audio effect parameter adjustment after the audio path test is completed. Testers can use **natural human voice** to speak to the remote device (such as a mobile phone or another intercom device) in an independent space as much as possible, and then use the remote device to listen to whether there is their own voice. If not, it means that the effect of echo suppression under far-end single talk is relatively ideal; otherwise, there will be residual echo transmitted back from the device, which requires further parameter adjustment and optimization.

- **Near-end speaks, far-end does not speak**

In this case, the system is in **near-end single talk** state, and the far-end does not speak (there is no sound in the local speaker), so there is no echo. At this time, it is actually equivalent to that the microphone only collects the near-end speaker and the ambient sound.

- **Near-end and far-end speak at the same time**

In this case, the system is in **double talk** state, in which the adaptive filter is easy to diverge, resulting in a decrease in the echo cancellation strength. Therefore, at this time, it is more necessary to pay attention to whether the playback volume is too large. Excessive echo energy causes the near-end microphone to re-collect and cause distortion.

Therefore, when testing the echo cancellation effect, the above three situations need to be tested in turn and the results of each round of testing should be paid attention to. If you only need to do the noise reduction algorithm, you can only focus on the test scenario of **near-end talking, far-end not talking**.

10.1.4 Audio channel test

Before debugging the algorithm, we need to test the audio channel, adjust the appropriate recording and playback gain, and provide the best electrical signal data to the algorithm to obtain a more ideal processing effect.

10.1.4.1 Audio playback maximum undistorted signal test

If a product has the need for intercom, we should first clarify the playback power and load of the speaker, and do a good job of calibrating the gain of the maximum undistorted signal. Otherwise, the distortion of the speaker will cause serious leakage echo anomalies.

Therefore, we can play a 0dBFS 1kHz sine wave, and then measure the voltage amplitude of the power amplifier (PA) output or the two ends of the speaker through an oscilloscope to determine whether the actual maximum undistorted output voltage meets the product positioning requirements. However, since PA is usually driven and amplified by ClassD, directly measuring the PA output or the two ends of the speaker will contain high-frequency carrier signals of several hundred kHz. Therefore, as shown in the figure below, we can connect RC resistors at both ends of the speaker to form a first-order filter circuit. The ratio of resistance and capacitance can be 500ohm/10nF, and the cut-off frequency is about 32kHz, which can filter out high-frequency carrier signals and is close to the upper limit of the human ear's audible frequency range of 20kHz.

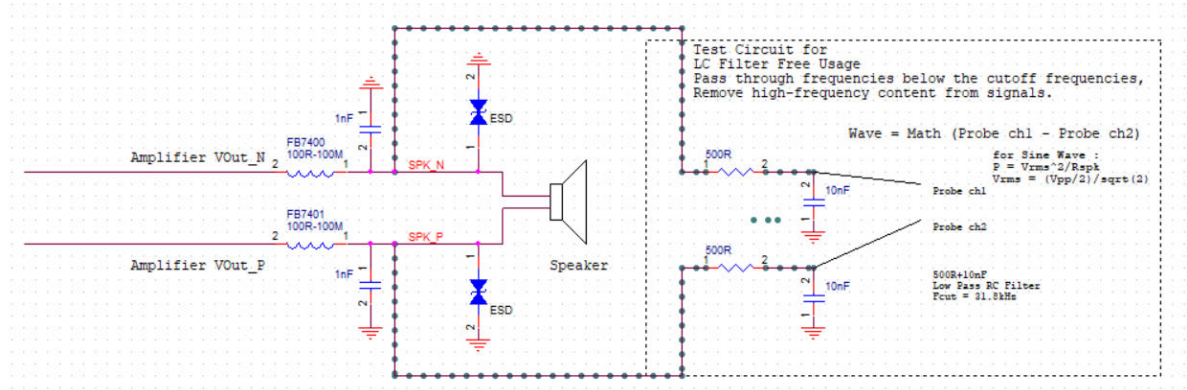


Figure How to measure the output amplitude of the speaker and add an RC filter circuit

The following command can be used to play a 0dBFS 1kHz sine wave. Note that the original data is played and the AO VQE algorithm cannot be turned on:

```
rk_mpi_ao_test -i /data/sine1000_16khz_2ch.wav --sound_card_name=hw:0,0 --
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

If you have tinyalsa or alsa-utils, you can also use the following command to play the test:

```
tinyplay /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -b 2 -D 0
```

```
aplay -Dhw:0,0 /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -f S16_LE
```

After some playback gain adjustment, we expect to measure the following signal at the speaker end:



Figure Measured 0dBFS 1kHz sine wave maximum undistorted voltage after RC filtering

The yellow and green waveforms are the P and N ends of the speaker respectively, and the pink one is the differential voltage amplitude obtained by the oscilloscope after math sub operation. If the rated power of a product design speaker is up to 1.5W@8ohm, then from the pink peak-to-peak voltage of 9.8V, we can get:

$$P = V_{rms}^2 / R = ((V_{pp}/2)/\sqrt{2})^2 / R = ((9.8/2)/\sqrt{2})^2 / 8 \approx 1.5W$$

The measured power value just meets the design requirement of 1.5W.

10.1.4.2 Audio sweep recording and playback test

First, configure the capture path. If you use the Rockit framework. Because the default output of rk_mpi_ai_test /tmp/cap_out.pcm has a size limit of 1MB, if you need to record for a long time, you can execute the following command to export the original audio data to /tmp/dbg_capin.pcm:

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm
```

Then execute the playback command without VQE algorithm:

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --
out_rate 16000 --out_ch 2 --output /tmp
```

At the same time, the machine needs to play a 0dBFS sweep file:

```
rk_mpi_ao_test -i /data/sweep_16k_20_8000_16b_2ch.wav --sound_card_name=hw:0,0 --
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

In this way, the original recording data before algorithm processing can be obtained:

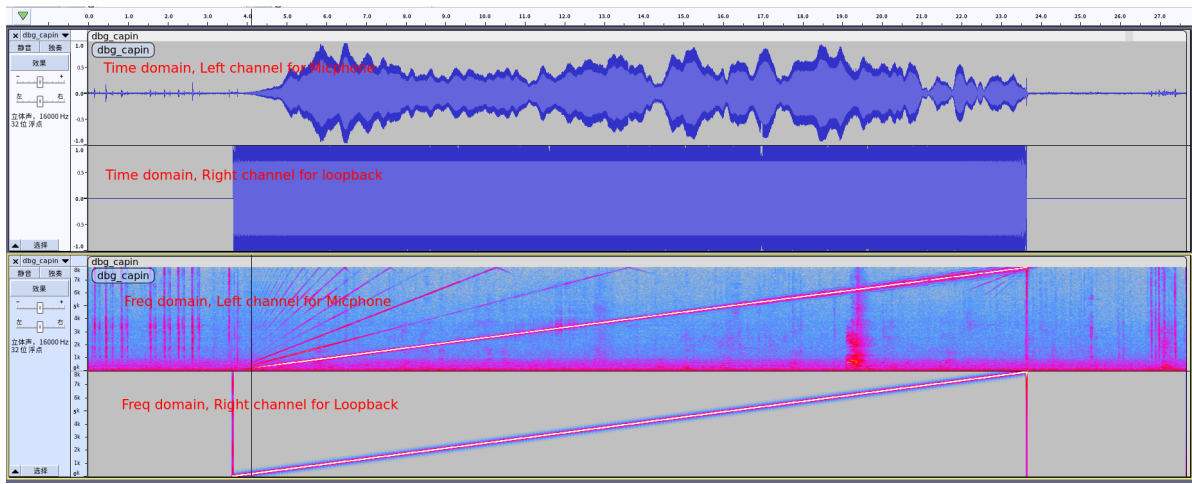


Figure Self-recording and self-playing sweep test

As can be seen in the figure above, the recorded audio data is 2ch, and the time domain and frequency domain status of 2ch are displayed respectively. The left channel is the data collected by the microphone in real time, and the right channel is the audio data collected back to the time just played, and the PCM content of the digitally collected back is consistent with the PCM content played out.

If the speaker gain is too large, the data recorded by the MIC may be measured to have clipping anomalies when playing 0dBFS sweep frequency:

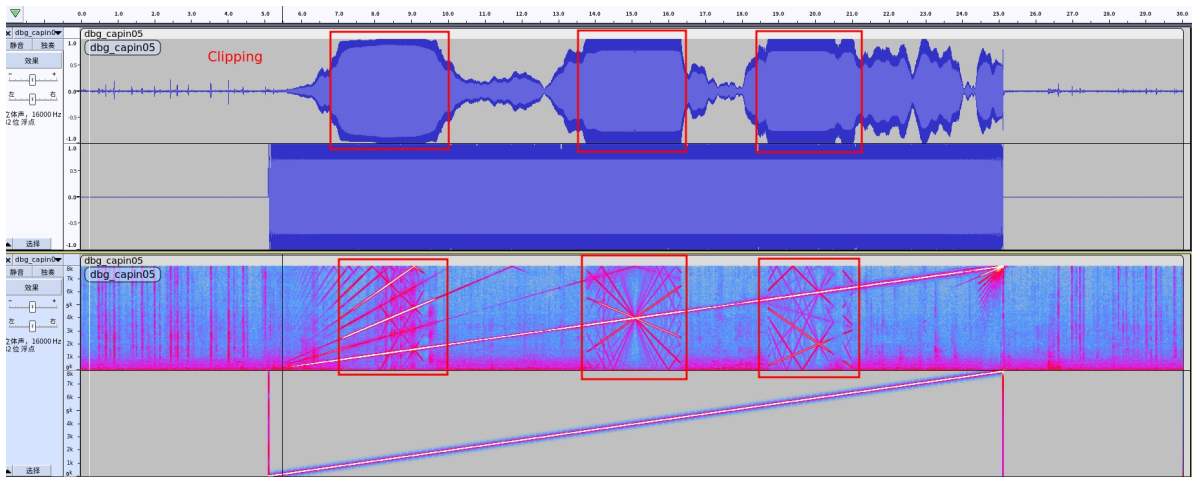


Figure Self-recording and self-playing sweep test clipping anomalies

You can zoom in on the time axis and see that the red frame part of the figure above has obvious clipping distortion. The distorted signal is an irreversible anomaly for the input of the back-end algorithm, which will affect the result of the algorithm processing.

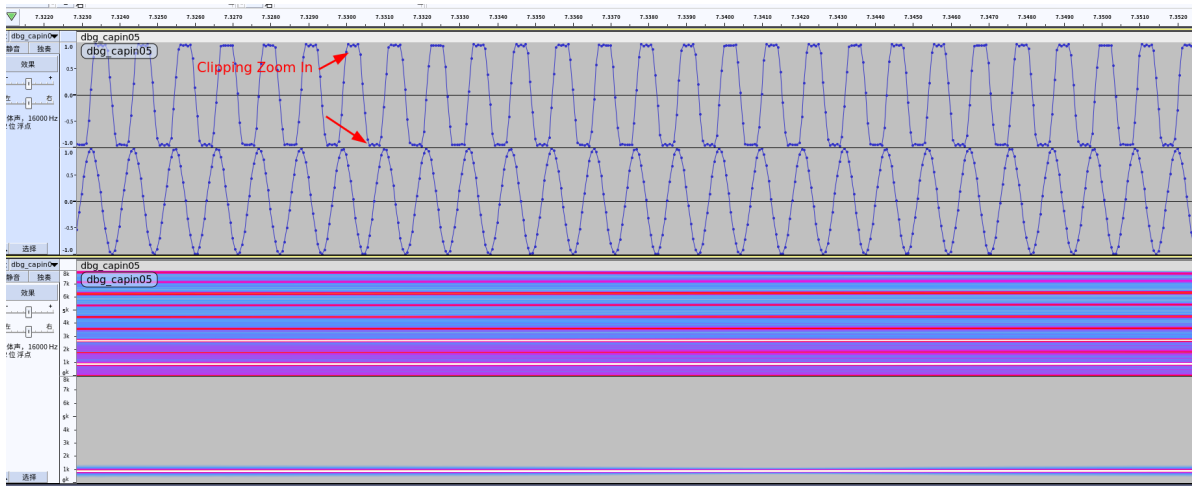


Figure Schematic diagram of abnormal clipping in self-recording and self-broadcasting sweep test

Therefore, if you encounter recording clipping distortion in this case, it is recommended to reduce the front-end codec analog gain first.

10.1.5 Audio algorithm debugging

Based on the above audio path test, we should have obtained relatively ideal input signal data. Before applying the integrated algorithm, we can first confirm whether the test board contains the algorithm library and AI VQE JSON configuration parameters. Their default paths in the released SDK are:

```
/oem/usr/lib/librkaudio_vqe.so
/oem/usr/lib/librkaudio_common.so
/oem/usr/share/vqefiles/config_aivqe.json
```

Then, before the application is executed, we configure some debugging-related environment variables to generate the information we need:

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm // Compatible with old versions of
rockit, export the recorded data before algorithm processing
export rt_debug_skv_out_file=/tmp/dbg_skvout.pcm // Compatible with old versions
of rockit, export the recorded data after AI VQE algorithm processing
export rt_debug_skv_ao_out_file=/tmp/dbg_ao_skvout.pcm // Compatible with old
versions of rockit, export the recorded data after AO VQE algorithm processing
export rt_debug_playback_file=/tmp/dbg_playback_out.pcm // Compatible with old
versions of rockit, export all the recorded data after AO algorithm processing,
which should be the same as the digital playback signal
export rt_dump_skv_param=true // Universal for rockit versions, print the
parameter configuration of AI VQE algorithm when running
export RKAUDIO_SKV_DUMP_CONF=true // Used by the new version of the algorithm
library, print the parameter configuration of the AI VQE algorithm when it is
running
export rt_dump_skv_inout=true // Used by the new version of rockit, export the
recording data before and after the AI VQE algorithm is processed, and generate
it to the /tmp directory by default. For example: /tmp/dbg-skv-in-2ch-16000.pcm
and /tmp/dbg-skv-out-1ch-16000.pcm
export rt_dump_skv_ao_inout=true // Used by the new version of rockit, export the
recording data before and after the AO VQE algorithm is processed, and generate
it to the /tmp directory by default. For example: /tmp/dbg-skv-ao-in-2ch-
16000.pcm and /tmp/dbg-skv-ao-out-1ch-16000.pcm
```

You can use the following command to enable the AI VQE algorithm, and the output is 1ch, so here '--out_ch 1':

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --
out_rate 16000 --out_ch 1 --output /tmp --vqe_enable 1 --vqe_cfg
/oem/usr/share/vqefiles/config_aivqe.json
```

At the same time, you can play the remote voice audio stored locally:

```
rk_mpi_ao_test -i /userdata/remote_voice.wav --sound_card_name=h
w:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```


If the playback also needs to use the VQE algorithm, you can also add the configuration parameters to enable AO VQE:

```
rk_mpi_ao_test -i /userdata/remote_voice.wav --sound_card_name=h
w:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2 --vqe_cf
g /oem/usr/share/vqefiles/config_aovqe.json --vqe_enable=1
```

In this way, when debugging, you can print out the above algorithm parameter information and the data generated before and after the AI/AO algorithm, put it on the computer and use the audacity tool to view it, mark the timeline, and locate the areas that need to be optimized.

10.2 Firmware size

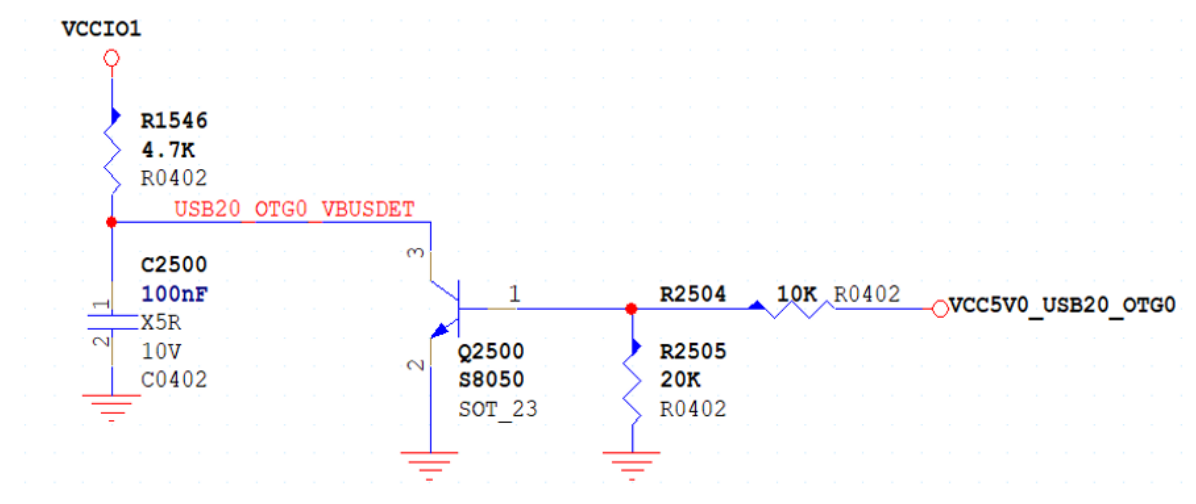
For the trim of the firmware size, refer to Section 8.5 of the document to select different configuration combinations to get the corresponding size of the firmware. The trim configuration can be carried out from three parts: U-Boot, Kernel, and Buildroot.

10.3 OTG-ID and OTG-VBUS-DET configuration Description

The 'usbphy' of the RK3506G simplifies the OTG-ID and OTG-VBUS pins, and can be simulated with GPIO to achieve related functions.

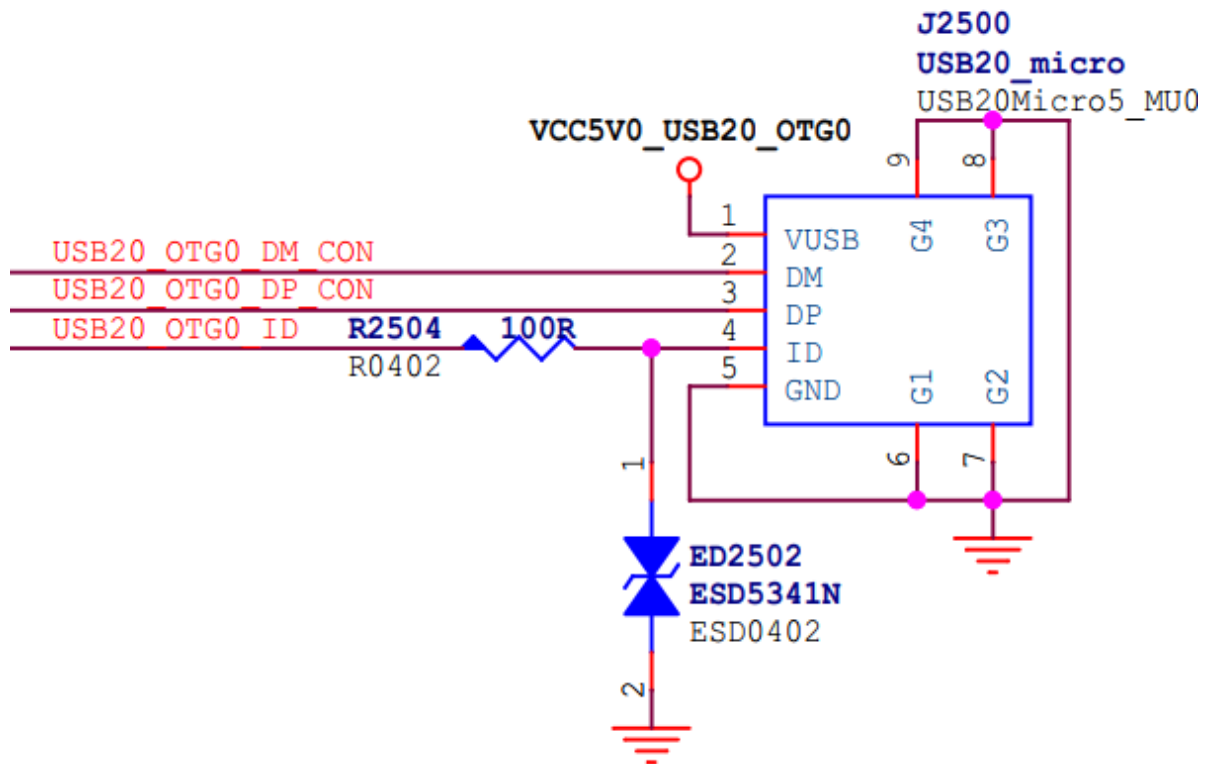
10.3.1 The OTG ID configure

OTG-ID is used to automatically switch between USB PERIPHERAL and USB HOST modes.



USB20_OTG0_ID connects to a GPIO, which needs to be pulled up internally.

OTG-VBUS-DET is used to detect USB DEVICE insertion and removal.



USB20_OTG0_VBUSDET Connects to the GPIO. VCCIO1 is the power supply for the GPIO power domain.

VCC_5V0_USB20_OTG0 Connects to the power supply of the USB bus.

The above two functions can be used in combination.

The DTS configuration is as follows:

```

/ {
    extcon_usb: extcon-usb {
        compatible = "linux,extcon-usb-gpio";
        vbus-gpio = <&gpio3 RK_PB2 GPIO_ACTIVE_LOW>;    /* Define vbus-det-pin */
        id-gpio = <&gpio3 RK_PB4 GPIO_ACTIVE_HIGH>;      /* Define id-det-pin */
        pinctrl-names = "default";
        pinctrl-0 = <&usb_extcon_vbus &usb_extcon_id>;    /* Reference pin's iomux */
    };

    status = "okay";
};

&u2phy_otg0 {
    vbus-supply = <&vcc5v0_otg0>;
    rockchip,gpio-id-det;                                /* Accept id-det
notifications */
    rockchip,gpio-vbus-det;                              /* Accept vbus-det
notifications */
    status = "okay";
};

&usb2phy {
    extcon = <&extcon_usb>;                                /* Notifications
referencing extcon_usb */
    status = "okay";
};

```



```

&pinctrl {
    usb {
        usb_extcon_id: usb-extcon-id {
            rockchip,pins = <3 RK_PB4 RK_FUNC_GPIO &pcfg_pull_up>;    /* The id
pin must be pulled up */
        };

        usb_extcon_vbus: usb-extcon-vbus {
            rockchip,pins = <3 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};

```

10.4 Video software decoding

The video soft solution uses CPU for decoding. The multimedia VDEC module has automatically selected hardware decoding or software decoding according to the chip type, and the related interfaces are consistent, so users do not need to be aware of the difference. The RKADK player in the SDK provides the corresponding VDEC creation and invocation methods.

The RKADK API reference documentation path is as follows:

```
<SDK>/docs/en/Linux/Multimedia/Rockchip_Developer_Guide_Linux_RKADK_EN.pdf
```

At present, the video soft solution only supports H264 decoding, and the video soft solution supports the output of the following image formats:

```

RK_FMT_YUV420P
RK_FMT_YUV420SP

```

Video soft decoding performance:

Test scenario	Frame rate
CPU frequency: 1.3GHz, resolution ratio: 1280x720, bit rate: 2862 kb/s (1 refs)	26
CPU frequency: 1.3GHz, resolution ratio: 720x512, bit rate: 1136 kb/s (1 refs)	59
CPU frequency: 1.5GHz, resolution ratio: 1280x720, bit rate: 2862 kb/s (1 refs)	30
CPU frequency: 1.5GHz, resolution ratio: 1280x720, bit rate: 1136 kb/s (1 refs)	72

This can be tested by the following command:

```

export send_data_debug_level=1
rkadk_player_test -i "/oem/720x512.mp4" -v -a 1 -s 0 -I 1 -P 0

```

10.5 MCU

RK3506 MCU only provide Bare-Metal support, the SDK automatic compilation of its configuration file is used to specify the compilation of CPU type (AP/MCU), such as modified device/rockchip/rk3506/amp_linux.its, adding MCU nodes

```
/dts-v1/;
/ {
    description = "FIT source file for rockchip AMP";
    #address-cells = <1>;

    images {
        amp2 {
            description = "rtos-core2";
            data = /incbin/("rtt2.bin");
            type = "firmware";
            compression = "none";
            arch = "arm"; // "arm64" or "arm"
            cpu = <0xf02>; // mpidr
            thumb = <0>; // 0: arm or thumb2; 1: thumb
            hyp = <0>; // 0: el1/svc; 1: el2/hyp
            load = <0x03e00000>;
            srambase = <0xffff80000>;
            sramsize = <0x0000c000>;
            compile {
                size = <0x00100000>;
                sys = "rtt";
                core = "ap";
                rtt_config = "board/evb1/defconfig";
            };
            udelay = <10000>;
            hash {
                algo = "sha256";
            };
        };

        mcu {
            description = "mcu";
            data = /incbin/("mcu.bin");
            type = "standalone"; # must be "standalone"
            compression = "none";
            arch = "arm"; # "arm64" or "arm", the same as U-
Boot state
            load = <0xffff84000>;
            udelay = <1000000>;
            compile {
                size = <0x00800000>;
                sys = "hal";
                core = "mcu"; # CPU type: ap or mcu
            };
            hash {
                algo = "sha256";
            };
        };
    };
};
```

```

configurations {
    default = "conf";
    conf {
        description = "Rockchip AMP images";
        rollback-index = <0x0>;
        loadables = "amp2", "mcu";

        signature {
            algo = "sha256,rsa2048";
            padding = "pss";
            key-name-hint = "dev";
            sign-images = "loadables";
        };

        /* - run linux on cpu0
        * - it is brought up by amp(that run on U-Boot)
        * - it is boot entry depends on U-Boot
        */
        linux {
            description = "linux-os";
            arch = "arm";
            cpu = <0xf00>;
            thumb = <0>;
            hyp = <0>;
            load = <0x00900000>;
            udelay = <0>;
        };
    };
};

```

After the mcu node to be compiled is added to the its, the file configuration of the its used for compilation is specified in defconfig

```
RK_AMP_FIT_ITS="amp_linux.its"
```

It is also possible to go into hal and compile MCU firmware separately

```

cd hal/project/rk3506-mcu/GCC
make clean;make -j16
cd ..
./mkimage.sh
ls Image/amp.img -l
-rw-rw-r-- 1 zjh zjh 28672 Aug 22 09:31 Image/amp.img

```

The use of MCU configuration can be referred to the AMP development documentation:

```
SDK/docs/en/Common/AMP/Rockchip_Developer_Guide_AMP_EN.pdf
```

10.6 Flexbus(DVP, High Speed IO)

Please refer to flexbus functions:

```
SDK/docs/cn/Common/FLEXBUS
```

10.7 The RS485 hardware scheme automatically switches

About RS485 hardware automatic switching solution, please refer to the following document to add the attribute:

```
linux, RS485-enabled -at-boot-time;
```

Reference document:

```
<SDK>/kernel/Documentation/devicetree/bindings/serial/rs485.yaml
```

10.8 Real-time optimization

Isolate CPU1 and add stress on the other CPUs:

```
stress-ng -c 3 --io 2 --vm 1 --vm-bytes 4M --timeout 1000000s
```

Disable Display

```
killall rk_demo
```

PREEMPT_RT

```
root@rk3506-buildroot:/# cyclicttest -m -a 1 -p 99 -t 1 -i 1000 --mainaffinity=0 -
D 2h
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 7.19 7.17 7.22 7/137 648

T: 0 ( 1117) P:99 I:1000 C:7199996 Min:      0 Act:      2 Avg:      3 Max:      62
(Testing for 2 hours)
```

Xenomai Cobalt Mode

```
root@rk3506-buildroot:/# cyclicttest -m -a 1 -p 99 -t 1 -i 1000 -D 2h
WARN: stat /dev/cpu_dma_latency failed: No such file or directory
policy: fifo: loadavg: 6.53 6.52 6.57 8/101 1115

T: 0 ( 1503) P:99 I:1000 C:7199992 Min:      4 Act:     15 Avg:     16 Max:     68
(Testing for 2 hours)
```

Patch and Documentation

```
docs/Patches/Real-Time-Performance/  
├─ PREEMPT_RT  
├─ Rockchip_Develop_Guide_Linux_RealTime_CN.md  
├─ Rockchip_Develop_Guide_Linux_RealTime_Performance_Test_Report_CN.pdf  
└─ XENOMAI
```

10.9 EtherCat

Please refer to EtherCat documentation:

```
<SDK>/docs/en/Linux/ApplicationNote/Rockchip_Use_Guide_Linux_EtherCAT_IgH_CN.pdf
```

10.10 CAN

Please refer to CAN documentation:

```
<SDK>/docs/en/Common/CAN/Rockchip_Developer_Guide_CAN_FD_EN.pdf  
<SDK>/docs/en/Common/CAN/Rockchip_Developer_Guide_Can_EN.pdf
```

10.11 DSMC

Please refer to the DSMC documentation:

```
<SDK>/docs/en/Common/DSMC
```

10.12 LVGL Demo

RK3506 use RK_DEMO by default, RK_DEMO include smart home, furniture control, intercom and system setting. The configurations of RK_DEMO are included in rockchip_rk3506_defconfig:

```
...  
#include "lvgl/lvgl_rkadk.config"  
#include "lvgl/rk_demo.config"  
...
```

The others configurations of LVGL please refer to

```
<SDK>/docs/cn/Linux/Graphics/Rockchip_Developer_Guide_Linux_LVGL_CN.pdf.
```

10.12.1 Enable Functions of RK_DEMO

Enter the buildroot menuconfig, search for and switch the macros as follows to enable or disable functions:

```
# Enable multimedia support, such as local video playback, rtsp video playback
and voice intercom
BR2_RK_DEMO_ENABLE_MULTIMEDIA
# Enable sensors
BR2_RK_DEMO_ENABLE_SENSOR
# Enable WiFi and BT functions
BR2_RK_DEMO_ENABLE_WIFIBT
# Enable ASR function
BR2_RK_DEMO_ENABLE_ASR
```

10.13 Display

RK3506 supports RGB, MIPI DSI and other display interfaces, display part of the document please refer to:

```
<SDK>/docs/en/Common/DISPLAY/
```

10.14 Read-write file system

At present, RK3506 supports a variety of file systems, and most products use small memory. By default, rootfs is packaged in ubi_squashfs type, and rootfs is read-only, which can reduce the space used and the cost. If you need to read and write file systems, you need to modify the rootfs packaging method as follows:

Modify rootfs type in the kernel:

```
diff --git a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi b/arch/arm/boot/dts/rk3506-
evb1-v10.dtsi
index 77f19b4c05c6..e2fe2f4c7c83 100644
--- a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
@@ -12,7 +12,7 @@ / {
    compatible = "rockchip,rk3506-evb1-v10", "rockchip,rk3506";

    chosen {
-       bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
ubi.mtd=5 ubi.block=0,rootfs root=/dev/ubiblock0_0 rootfstype=squashfs rootwait
snd_aloop.index=7 snd_aloop.use_raw_jiffies=1";
+       bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
ubi.mtd=5 root=ubi0:rootfs rw rootfstype=ubifs rootwait snd_aloop.index=7
snd_aloop.use_raw_jiffies=1";
    };

    acodec_sound: acodec-sound {
```

Modify the configuration file in Buildroot:

```

diff --git a/configs/rockchip_rk3506_defconfig
b/configs/rockchip_rk3506_defconfig
index 0908a3e48f..cd5e1786b2 100644
--- a/configs/rockchip_rk3506_defconfig
+++ b/configs/rockchip_rk3506_defconfig
@@ -8,6 +8,7 @@
#include "wifibt/wireless.config"
#include "lvgl/lvgl_rk3506.config"
#include "lvgl/rk_demo.config"
+include "fs/ubifs.config"
# BR2_PACKAGE_CHRONY is reset to default
# BR2_PACKAGE_DNSMASQ is reset to default
# BR2_PACKAGE_DROPBEAR is reset to default
@@ -24,7 +25,7 @@ BR2_ROCKCHIP_ALSA_CONFIG_INSTALL_INIT_SCRIPT=y
# BR2_TARGET_GENERIC_REMOUNT_ROOTFS_RW is not set
BR2_TARGET_ROOTFS_SQUASHFS4_ZSTD=y
BR2_TARGET_ROOTFS_UBI=y
-BR2_TARGET_ROOTFS_UBI_SQUASHFS=y
+BR2_TARGET_ROOTFS_UBI_UBIFS=y
# BR2_TARGET_ROOTFS_UBI_SQUASHFS_STATIC is not set
BR2_TARGET_ROOTFS_UBI_SUBSIZE=2048
BR2_TIME_BITS_64=y

```

Note After the file system is changed to a read-write file system, the file system mirror will become larger, and the partition table needs to be adjusted accordingly.

10.15 How to use eMMC Storage Media

By default, all storage media in the SDK are SPI NAND. If you want to use eMMC storage media, you need to make the following changes:

SDK/device/rockchip/rk3506/parameter-128M.txt

```

diff --git a/.chips/rk3506/parameter-128M.txt b/.chips/rk3506/parameter-128M.txt
index 8147cd5..3c2ba99 100644
--- a/.chips/rk3506/parameter-128M.txt
+++ b/.chips/rk3506/parameter-128M.txt
@@ -9,5 +9,5 @@ CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0
-
CMDLINE:mtddparts=:0x00002000@0x00002000 (uboot),0x00000800@0x00004000 (misc),0x0000
0200@0x00004800 (vnvm),0x00007000@0x00004a00 (recovery),0x00005000@0x0000ba00 (boot)
,0x00020000@0x00010a00 (rootfs),0x00008000@0x00030a00 (oem),-
@0x00038a00 (userdata:grow)
+CMDLINE:mtddparts=:0x00002000@0x00004000 (uboot),0x00002000@0x00006000 (misc),0x000
20000@0x00008000 (boot),0x00040000@0x00028000 (recovery),0x00010000@0x00068000 (back
up),0x00c00000@0x00078000 (rootfs),0x00040000@0x00c78000 (oem),-
@0x00cb8000 (userdata:grow)
uid:rootfs=614e0000-0000-4b53-8000-1d28000054a9
diff --git a/.chips/rk3506/rockchip_rk3506_b_evbl_defconfig
b/.chips/rk3506/rockchip_rk3506_b_evbl_defconfig

```

```

index e1b6d3f..dc000cf 100644
--- a/.chips/rk3506/rockchip_rk3506_b_evbl_defconfig
+++ b/.chips/rk3506/rockchip_rk3506_b_evbl_defconfig
@@ -1,20 +1,17 @@
-RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG="rk3506"
-RK_UBOOT_CFG_FRAGMENTS="rk3506b rk3506_tb"
+RK_UBOOT_CFG_FRAGMENTS="rk3506b rk-emmc"
RK_UBOOT_SPL=y
RK_KERNEL_ARM32=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506b-evbl-v10"
RK_BOOT_COMPRESSED=y
-RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
-RK_FLASH_SIZE=2048
-RK_EXTRA_PARTITION_1_FSTYPE="ubi"
+RK_EXTRA_PARTITION_1_FSTYPE="ext2"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
-RK_EXTRA_PARTITION_2_FSTYPE="ubi"
+RK_EXTRA_PARTITION_2_FSTYPE="ext2"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y

```

SDK/u-boot:

```

From aff710b40d23e8d2b8b76f16f7d967bf7d4af82e Mon Sep 17 00:00:00 2001
From: Xuhui Lin <xuhui.lin@rock-chips.com>
Date: Tue, 22 Oct 2024 14:17:44 +0800
Subject: [PATCH] rockchip: rk3506: Add unset mmc iomux support

```

Because rk3506 evb boards don't have sdmmc_det by default,
remove mmc support in rk3506_defconfig.

If mmc support is needed, please:

- (1) make rk3506_defconfig rk-emmc.config
- (2) ./make.sh --spl-new

```

Change-Id: I626f49ba069bd4739a441c7fa677cccb041280a8
Signed-off-by: Xuhui Lin <xuhui.lin@rock-chips.com>

```

```

arch/arm/dts/rk3506-u-boot.dtsi          |  2 +-
arch/arm/mach-rockchip/rk3506/rk3506.c   | 70 ++++++-----
configs/rk-emmc.config                    |  5 ++
configs/rk3506_defconfig                  |  5 +-
4 files changed, 60 insertions(+), 22 deletions(-)

```

```

diff --git a/arch/arm/dts/rk3506-u-boot.dtsi b/arch/arm/dts/rk3506-u-boot.dtsi
index 2b61487..7096f34 100644
--- a/arch/arm/dts/rk3506-u-boot.dtsi
+++ b/arch/arm/dts/rk3506-u-boot.dtsi

```



```
@@ -11,7 +11,7 @@
```

```
    chosen: chosen {
        stdout-path = &uart0;
-       u-boot,spl-boot-order = &spi_nand, &spi_nor;
+       u-boot,spl-boot-order = &mmc, &spi_nand, &spi_nor;
    };
};
```

```
diff --git a/arch/arm/mach-rockchip/rk3506/rk3506.c b/arch/arm/mach-rockchip/rk3506/rk3506.c
```

```
index a147f54..cf919cf 100644
```

```
--- a/arch/arm/mach-rockchip/rk3506/rk3506.c
```

```
+++ b/arch/arm/mach-rockchip/rk3506/rk3506.c
```

```
@@ -5,6 +5,7 @@
```

```
    */
#include <common.h>
#include <dm.h>
+#include <spl.h>
#include <asm/io.h>
#include <asm/arch/cpu.h>
#include <asm/arch/hardware.h>
@@ -45,23 +46,6 @@ void board_debug_uart_init(void)
    /* No need to change uart*/
}

-#ifndef CONFIG_SPL_BUILD
-void rockchip_stimer_init(void)
-{
-    /* If Timer already enabled, don't re-init it */
-    u32 reg = readl(CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
-    if (reg & 0x1)
-        return;
-    writel(0x00010000, CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
-
-    asm volatile("mcr p15, 0, %0, c14, c0, 0"
-        : : "r"(COUNTER_FREQUENCY));
-    writel(0xffffffff, CONFIG_ROCKCHIP_STIMER_BASE + 0x14);
-    writel(0xffffffff, CONFIG_ROCKCHIP_STIMER_BASE + 0x18);
-    writel(0x00010001, CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
-}
-#endif
-
void board_set_iomux(enum if_type if_type, int devnum, int routing)
{
    switch (if_type) {
@@ -83,6 +67,55 @@ void board_set_iomux(enum if_type if_type, int devnum, int routing)
    }
}

+void board_unset_iomux(enum if_type if_type, int devnum, int routing)
+{
+    switch (if_type) {
+    case IF_TYPE_MMC:
```

```

+     if (devnum == 0) {
+         writel(0xffff0000, GPIO3_IOC_BASE + GPIO3A_IOMUX_SEL_0);
+         writel(0x00ff0022, GPIO3_IOC_BASE + GPIO3A_IOMUX_SEL_1);
+     }
+     break;
+ case IF_TYPE_MTD:
+     if (routing == 0) {
+         /* FSPI M0 */
+         writel(0xffff0000, GPIO2_IOC_BASE + GPIO2A_IOMUX_SEL_0);
+         writel(0x00ff0000, GPIO2_IOC_BASE + GPIO2A_IOMUX_SEL_1);
+     }
+     break;
+ default:
+     break;
+ }
+}
+
+#ifdef CONFIG_SPL_BUILD
+void rockchip_stimer_init(void)
+{
+    /* If Timer already enabled, don't re-init it */
+    u32 reg = readl(CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
+    if (reg & 0x1)
+        return;
+    writel(0x00010000, CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
+
+    asm volatile("mcr p15, 0, %0, c14, c0, 0"
+        : : "r"(COUNTER_FREQUENCY));
+    writel(0xffffffff, CONFIG_ROCKCHIP_STIMER_BASE + 0x14);
+    writel(0xffffffff, CONFIG_ROCKCHIP_STIMER_BASE + 0x18);
+    writel(0x00010001, CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
+}
+
+void spl_board_storages_fixup(struct spl_image_loader *loader)
+{
+    if (!loader)
+        return;
+
+    if (loader->boot_device == BOOT_DEVICE_MMC1 ||
+        loader->boot_device == BOOT_DEVICE_MMC2)
+        /* Unset the sdmmc0 iomux */
+        board_unset_iomux(IF_TYPE_MMC, 0, 0);
+}
+
+#endif
+
+int arch_cpu_init(void)
+{
+    #if defined(CONFIG_SPL_BUILD)
@@ -92,9 +125,10 @@ int arch_cpu_init(void)
+        val = readl(FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
+        writel(val & 0xffff00ff, FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
+
+    #if defined(CONFIG_MMC_DW_ROCKCHIP)
+        /* Set the sdmmc/emmc iomux */
+        board_set_iomux(IF_TYPE_MMC, 0, 0);

```

```

-
+endif
    /* Set the fspi to access ddr memory */
    val = readl(FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
    writel(val & 0xff00ffff, FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
diff --git a/configs/rk-emmc.config b/configs/rk-emmc.config
index 5e7dad3..bf9548b 100644
--- a/configs/rk-emmc.config
+++ b/configs/rk-emmc.config
@@ -1,2 +1,7 @@
    CONFIG_ROCKCHIP_EMMC_IOMUX=y
    # CONFIG_ROCKCHIP_SFC_IOMUX is not set
+CONFIG_CMD_MMC=y
+CONFIG_MMC=y
+CONFIG_DM_MMC=y
+CONFIG_MMC_DW=y
+CONFIG_MMC_DW_ROCKCHIP=y
diff --git a/configs/rk3506_defconfig b/configs/rk3506_defconfig
index c473a55..1ee0e96 100644
--- a/configs/rk3506_defconfig
+++ b/configs/rk3506_defconfig
@@ -57,7 +57,6 @@ CONFIG_RANDOM_UUID=y
    # CONFIG_CMD_LOADB is not set
    # CONFIG_CMD_LOADS is not set
    CONFIG_CMD_BOOT_ANDROID=y
-CONFIG_CMD_MMC=y
    CONFIG_CMD_MTD=y
    # CONFIG_CMD_ITEST is not set
    CONFIG_CMD_SCRIPT_UPDATE=y
@@ -100,8 +99,8 @@ CONFIG_MISC=y
    CONFIG_SPL_MISC=y
    CONFIG_ROCKCHIP_OTP=y
    # CONFIG_SUPPORT_EMMC_RPMB is not set
-CONFIG_MMC_DW=y
-CONFIG_MMC_DW_ROCKCHIP=y
+# CONFIG_MMC is not set
+# CONFIG_DM_MMC is not set
    CONFIG_MTD=y
    CONFIG_MTD_BLK=y
    CONFIG_MTD_DEVICE=y
--
2.34.1

```

SDK/kernel:

```

diff --git a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi b/arch/arm/boot/dts/rk3502-
evb1-v10.dtsi
index 8f4e57790..4795c7d0a 100644
--- a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
@@ -194,6 +194,16 @@ vcc3v3_stb: vcc3v3-stb {
        vin-supply = <&vcc_sys>;
    };

+    vcc_3v3: vcc-3v3 {

```

```

+         compatible = "regulator-fixed";
+         regulator-name = "vcc_3v3";
+         regulator-always-on;
+         regulator-boot-on;
+         regulator-min-microvolt = <3300000>;
+         regulator-max-microvolt = <3300000>;
+         vin-supply = <&vcc_sys>;
+     };
+
+     vcc_1v8: vcc-1v8 {
+         compatible = "regulator-fixed";
+         regulator-name = "vcc_1v8";
@@ -321,6 +331,7 @@ es8388: es8388@11 {
    };

    &mmc {
+    /*
+        bus-width = <4>;
+        cap-sd-highspeed;
+        no-sd;
@@ -332,6 +343,20 @@ &mmc {
+        non-removable;
+        mmc-pwrseq = <&sdio_pwrseq>;
+        sd-uhs-sdr104;
+    */
+
+    //For eMMC
+    no-sdio;
+    no-sd;
+    cap-mmc-highspeed;
+    cap-sd-highspeed;
+    non-removable;
+    mmc-hs200-1_8v;
+    pinctrl-names = "default";
+    pinctrl-0 = <&sdmmc_clk_pins &sdmmc_cmd_pins &sdmmc_bus4_pins>;
+    //vqmmc-supply = <&vccio4>;
+    //vmmc-supply = <&vcc_3v3>;
+
+    status = "okay";
    };

diff --git a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi b/arch/arm/boot/dts/rk3506-
evb1-v10.dtsi
index b1b182de7..1e0a47171 100644
--- a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
@@ -13,7 +13,7 @@ {
     compatible = "rockchip,rk3506-evb1-v10", "rockchip,rk3506";

     chosen {
-        bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0 ubi.mtd=5
ubi.block=0,rootfs root=/dev/ubiblock0_0 rootfstype=squashfs rootwait
snd_aloop.index=7 snd_aloop.use_raw_jiffies=1";

```

```
+     bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rootwait snd_aloop.index=7
snd_aloop.use_raw_jiffies=1";
};

backlight: backlight {
diff --git a/arch/arm/configs/rk3506_defconfig
b/arch/arm/configs/rk3506_defconfig
index d6cfd7e6..39fd1cc65 100644
--- a/arch/arm/configs/rk3506_defconfig
+++ b/arch/arm/configs/rk3506_defconfig
@@ -253,6 +253,9 @@ CONFIG_USB_GADGET=m
CONFIG_USB_CONFIGFS=m
CONFIG_USB_CONFIGFS_UEVENT=y
CONFIG_USB_CONFIGFS_F_FS=y
+CONFIG_MMC=y
+CONFIG_MMC_DW=y
+CONFIG_MMC_DW_ROCKCHIP=y
CONFIG_USB_ROLE_SWITCH=y
CONFIG_MMC=y
CONFIG_MMC_QUEUE_DEPTH=1
@@ -303,7 +306,8 @@ CONFIG_PWM_ROCKCHIP=y
CONFIG_PHY_ROCKCHIP_INNO_USB2=m
CONFIG_PHY_ROCKCHIP_INNO_DSIDPHY=y
CONFIG_NVMEM_ROCKCHIP_OTP=y
-CONFIG_EXT4_FS=m
+CONFIG_EXT2_FS=y
+CONFIG_EXT4_FS=y
# CONFIG_DNOTIFY is not set
CONFIG_VFAT_FS=m
CONFIG_EXFAT_FS=m
```